```
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from PIL import Image, ImageTk
import cv2
import numpy as np
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure

class FeatureDetectorApp(tk.Tk):

    def __init__(self):
        super().__init__()
        self.title("Interactive Feature Detection — Keypoint Detectors")
        self.geometry("1200x820")
        self.minsize(900, 600)

        self.original_image = None
        self.processed_image = None
        self.last_transform_text = tk.StringVar(value="Load an image to begin…")
        self._resize_job = None

        self.setup_ui()

    # ---------------------------- UI LAYOUT
    def setup_ui(self):
        # === IMAGE PANES ===
        top = tk.Frame(self, bg="#2c3e50")
        top.pack(side="top", padx=10, pady=(10, 6), fill="both", expand=True)

        pane = tk.PanedWindow(top, orient=tk.HORIZONTAL, sashrelief='raised',
bg="#2c3e50")
        pane.pack(fill="both", expand=True)

        left_panel = tk.Frame(pane, bg="#2c3e50", width=600, height=360)
        right_panel = tk.Frame(pane, bg="#2c3e50", width=600, height=360)
        pane.add(left_panel)
        pane.add(right_panel)

        self.orig_label = tk.Label(left_panel, bg="#2c3e50")
        self.orig_label.pack(fill="both", expand=True)
        self.proc_label = tk.Label(right_panel, bg="#2c3e50")
        self.proc_label.pack(fill="both", expand=True)

        # === HISTOGRAMS ===
        mid = tk.Frame(self, bg="#243447")
```

```
        mid.pack(side="top", padx=10, pady=(6, 6), fill="x")

        self.fig = Figure(figsize=(10, 2.8), dpi=100)
        self.ax_orig = self.fig.add_subplot(1, 2, 1)
        self.ax_proc = self.fig.add_subplot(1, 2, 2)
        self.ax_orig.set_title("Original Histogram")
        self.ax_proc.set_title("Keypoint Overlay Histogram")

        self.canvas = FigureCanvasTkAgg(self.fig, master=mid)
        self.canvas.get_tk_widget().pack(fill="x", expand=False)

        # === BOTTOM CONTROLS ===
        bottom = tk.Frame(self, bg="#34495e", padx=10, pady=10)
        bottom.pack(side="bottom", fill="x")

        # === FILE BUTTONS ===
        file_frame = tk.Frame(bottom, bg="#34495e")
        file_frame.grid(row=0, column=0, sticky="w", padx=(0, 10))
        tk.Button(file_frame, text="Open Image…", command=self.open_image,
                  bg="#1abc9c", fg="white", font=("Helvetica", 12,
"bold")).pack(side="left", padx=5)
        tk.Button(file_frame, text="Reset", command=self.reset_image,
                  bg="#e74c3c", fg="white", font=("Helvetica", 12,
"bold")).pack(side="left", padx=5)
        tk.Button(file_frame, text="Save As…", command=self.save_image,
                  bg="#9b59b6", fg="white", font=("Helvetica", 12,
"bold")).pack(side="left", padx=5)

        # === DETECTOR BUTTONS ===
        trans_container = tk.Frame(bottom, bg="#34495e")
        trans_container.grid(row=0, column=1, sticky="ew")
        bottom.grid_columnconfigure(1, weight=1)

        trans_canvas = tk.Canvas(trans_container, bg="#34495e",
highlightthickness=0, height=140)
        trans_scroll = ttk.Scrollbar(trans_container, orient="horizontal",
command=trans_canvas.xview)
        self.trans_inner = tk.Frame(trans_canvas, bg="#34495e")
        self.trans_inner.bind("<Configure>", lambda e:
trans_canvas.configure(scrollregion=trans_canvas.bbox("all")))
        trans_canvas.create_window((0, 0), window=self.trans_inner, anchor="nw")
        trans_canvas.configure(xscrollcommand=trans_scroll.set)
        trans_canvas.pack(fill="x", expand=True)
        trans_scroll.pack(fill="x", expand=False)
```

```
        # === Feature Detectors ===
        feat_frame = tk.LabelFrame(self.trans_inner, text="Keypoint Detectors",
bg="#34495e", fg="white")
        feat_frame.pack(side="left", padx=6)

        detectors = [
            ("SIFT", "sift"),
            ("SURF", "surf"),
            ("BRISK", "brisk"),
            ("FAST", "fast")
        ]

        for name, key in detectors:
            tk.Button(feat_frame, text=name,
                      command=lambda t=key: self.apply_detector(t),
                      bg="#2980b9", fg="white").pack(pady=2, padx=5, fill="x")

        # === Transformation Info ===
        report_frame = tk.Frame(self, bg="#22313f")
        report_frame.pack(fill="x", padx=10, pady=(6, 10))
        tk.Label(report_frame, text="Feature Detector Used:", bg="#22313f",
fg="white",
                 font=("Helvetica", 11, "bold")).pack(anchor="w")
        self.transform_label = tk.Label(report_frame,
textvariable=self.last_transform_text,
                                        bg="#22313f", fg="#ecf0f1",
justify="left",
                                        font=("Consolas", 10))
        self.transform_label.pack(fill="x")

        self.bind('<Configure>', self._on_resize)

    # ---------------------------- IO
    def open_image(self):
        filepath = filedialog.askopenfilename(
            title="Select an Image",
            filetypes=(("Image files", "*.jpg *.jpeg *.png *.bmp *.tiff"), ("All
files", "*.*"))
        )
        if not filepath:
            return
        img = cv2.imread(filepath)
        if img is None:
            messagebox.showerror("Error", "Could not load image.")
            return
```

```
        self.original_image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        self.processed_image = self.original_image.copy()
        self.update_preview_and_histograms()
        self.last_transform_text.set("Loaded image.")

    def save_image(self):
        if self.processed_image is None:
            messagebox.showwarning("No Image", "No processed image to save.")
            return
        save_path = filedialog.asksaveasfilename(
            title="Save Processed Image As",
            defaultextension=".png",
            filetypes=(("PNG", "*.png"), ("JPEG", "*.jpg;*.jpeg"), ("BMP",
"*.bmp"))
        )
        if save_path:
            cv2.imwrite(save_path, self.processed_image)
            messagebox.showinfo("Saved", f"Image saved to:\n{save_path}")

    def reset_image(self):
        if self.original_image is not None:
            self.processed_image = self.original_image.copy()
            self.update_preview_and_histograms()
            self.last_transform_text.set("Reset to original image.")

    # ---------------------------- FEATURE DETECTORS
    def apply_detector(self, name):
        if self.original_image is None:
            messagebox.showwarning("No Image", "Please load an image first.")
            return
        img = self.original_image.copy()
        color_img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

        try:
            if name == "sift":
                sift = cv2.SIFT_create()
                kp, des = sift.detectAndCompute(img, None)
                self.last_transform_text.set(f"SIFT detected {len(kp)}
keypoints.")

            elif name == "surf":
                surf = cv2.xfeatures2d.SURF_create(400)
                kp, des = surf.detectAndCompute(img, None)
                self.last_transform_text.set(f"SURF detected {len(kp)}
keypoints.")
```

```
            elif name == "fast":
                fast = cv2.FastFeatureDetector_create()
                kp = fast.detect(img, None)
                des = None
                self.last_transform_text.set(f"FAST detected {len(kp)}
keypoints.")

            elif name == "brisk":
                brisk = cv2.BRISK_create()
                kp, des = brisk.detectAndCompute(img, None)
                self.last_transform_text.set(f"BRISK detected {len(kp)}
keypoints.")

            else:
                return

            out = cv2.drawKeypoints(color_img, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
            self.processed_image = cv2.cvtColor(out, cv2.COLOR_BGR2RGB)

        except cv2.error as e:
            messagebox.showerror("Error", f"OpenCV Error:\n{e}")
            return

        self.update_preview_and_histograms()

    # ------------------------------ DISPLAY
    def _show_image_in_label(self, img, label):
        if img is None:
            label.configure(image='')
            return
        frame_w = label.winfo_width() or 400
        frame_h = label.winfo_height() or 300
        h, w = img.shape[:2]
        scale = min(frame_w / w, frame_h / h)
        resized = cv2.resize(img, (int(w * scale), int(h * scale)))
        if len(resized.shape) == 2:
            disp = Image.fromarray(resized)
        else:
            disp = Image.fromarray(cv2.cvtColor(resized, cv2.COLOR_RGB2BGR))
        img_tk = ImageTk.PhotoImage(image=disp)
        label.configure(image=img_tk)
        label.image = img_tk
```

```
    def update_histograms(self):
        self.ax_orig.clear(); self.ax_proc.clear()
        if self.original_image is not None:
            self.ax_orig.hist(self.original_image.ravel(), bins=256, range=(0,
255))
        if self.processed_image is not None:
            gray = cv2.cvtColor(self.processed_image, cv2.COLOR_RGB2GRAY) if
len(self.processed_image.shape) == 3 else self.processed_image
            self.ax_proc.hist(gray.ravel(), bins=256, range=(0, 255))
        self.fig.tight_layout()
        self.canvas.draw_idle()

    def update_preview_and_histograms(self):
        self._show_image_in_label(self.original_image, self.orig_label)
        self._show_image_in_label(self.processed_image, self.proc_label)
        self.update_histograms()

    def _on_resize(self, event):
        if self._resize_job:
            try: self.after_cancel(self._resize_job)
            except Exception: pass
        self._resize_job = self.after(180, self.update_preview_and_histograms)


if __name__ == "__main__":
    app = FeatureDetectorApp()
    app.mainloop()
```

BRISK 839 KEYPOINTS

FAST 1841 Keypoint

SIFT 596 Keypoint
